

一种字符串压缩表示的解压

题目描述

有一种简易 **压缩算法** ：针对全部由小写英文字母组成的字符串，将其中连续超过两个相同字母的部分压缩为连续个数加该字母，其他部分保持原样不变。

例如：字符串“aaabbcccd”经过压缩成为字符串“3abb4cd”。

请您编写解压函数，根据输入的字符串，判断其是否为合法压缩过的字符串，

若输入合法则输出解压缩后的字符串，否则 **输出字符串** “!error”来报告错误。

输入描述

输入一行，为一个ASCII字符串，长度不会超过100字符，用例保证输出的字符串长度也不会超过100字符。

输出描述

若判断输入为合法的经过压缩后的字符串，则输出压缩前的字符串；

若输入不合法，则输出字符串“!error”。

用例

输入	4dff
输出	ddddff
说明	4d扩展为dddd，故解压后的字符串为ddddff。

输入	2dff
输出	!error

说明	两个d不需要压缩，故输入不合法。
输入	4d@A
输出	!error
说明	全部由小写英文字母组成的字符串压缩后不会出现特殊字符@和大写字母A，故输入不合法。

题目解析

本题主要考察字符串操作。

我们遍历输入的压缩串的每一个字符 c：

- 若 c 是数字，则此时
 1. 继续往后读取连续的数字字符，比如 "41dff"，当遍历第一个字符 c = '4' 时，我们继续往后读取完连续的数字，得到一个重复次数41
 2. 读取完重复次数后，继续读取一个字符，看是否为小写字母，若不是，则输入串不合法，若是，比如 "41dff" 中 41 后面的 'd'，则需要将 'd' 重复41次后拼接到结果串。
- 若 c 是小写字母，则是一个独立的小写字母，直接拼接到结果串即可。
- 若 c 是其他字符，则输入串不合法。

按上面逻辑，直到遍历完输入串。即可得到一个解压后的结束串。

2023.05.25

给几个用例：3bb，bbb，3b4b

大家觉得这几个压缩字符串是合法的吗？

答案是不合法，原因很简单，我们将他们解压后，再按照题目要求重新压缩，得到的压缩字符串和这些是不一样的，比如3bb，解压后为bbbb，再次压缩后为4b。

因此，我们只能得到解压字符串后，还需要重新压缩，来对比输入的字符串，如果不一样，则说明输入的压缩字符串不合法，需要返回!error

2024.1.15

下面代码中，当我们用正则匹配到：数字+字母，比如4d，后面进行了字符串替换，将字符串中所有4d替换为了dddd，这是有问题的，比如输入串为 4d5a14d，进行4d替换后，就变为dddd5a1dddd，这是不正确的，因为后面的14d被错误替换为1dddd了，因此我们进行字符串替换操作时，只能进行首次替换。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const zipStr = await readline(); // 输入的压缩串
7      const unZipStr = unZip(zipStr); // 解压后字符串
8
9      if (unZipStr == "!error" || zipStr != zip(unZipStr)) { // 对解压后的字符串二次压缩，如果二次压缩结果和输入的压缩串不一致，则输入不合法
10         console.log("!error");
11     } else {
12         console.log(unZipStr);
13     }
14 })();
15
16 // 解压
17 function unZip(zipStr) {
18     const unZipStr = []; // 记录解压结果串
19
20     let i = 0;
21     while (i < zipStr.length) {
22         const c = zipStr[i];
23
24         if (isDigit(c)) {
```

运行

```
25     const num = []; 26 |
27     while (i < zipStr.length && isDigit(zipStr[i])) { // 如果遍历的字符是数字
28         // 则探索出之后连续的所有数字
29         num.push(zipStr[i]);
30         i++;
31     }
32
33     // 以及跟着的一个小写字母
34     if (i < zipStr.length && isLower(zipStr[i])) {
35         const repeatCount = parseInt(num.join(""));
36         const repeatLetter = zipStr[i];
37
38         // 必须超过连续两个相同小写字母才能进行压缩
39         if (repeatCount <= 2) return "!error";
40
41         // 解压
42         unZipStr.push(new Array(repeatCount).fill(repeatLetter).join(""));
43     } else {
44         // 数字后面没有跟着小写字母，则不合法
45         return "!error";
46     }
47 } else if (isLower(c)) {
48     // 单独的小写字母直接并入解压串
49     unZipStr.push(c);
50 } else {
51     // 压缩串中含有数字,小写字母以外的字符，则不合法
52     return "!error";
53 }
54
55 i++;
56 }
57
58 return unZipStr.join("");
59 }
60
61 // 压缩
62 function zip(unZipStr) {
63     const zipStr = [];
```

```
64 |
65 | // 加一个空格不影响压缩结果, 同时可以避免收尾操作
66 | unZipStr += " ";
67 |
68 | // 记录上一个字母, 以及它的出现次数
69 | let last = unZipStr[0];
70 | let count = 1;
71 |
72 | for (let i = 1; i < unZipStr.length; i++) {
73 |     const cur = unZipStr[i];
74 |
75 |     if (cur == last) {
76 |         // 当前字母和上一个字母相同, 则连续相同该字母数量++
77 |         count++;
78 |     } else {
79 |         // 当前字母和上一个字母不相同, 则连续相同字母被打断
80 |         if (count > 2) {
81 |             // 超过两个连续相同字母, 则进行压缩
82 |             zipStr.push(count);
83 |             zipStr.push(last);
84 |         } else {
85 |             // 不超过连续两个字母, 则不进行压缩
86 |             zipStr.push(new Array(count).fill(last).join(""));
87 |         }
88 |
89 |         // 更新last和count
90 |         last = cur;
91 |         count = 1;
92 |     }
93 | }
94 |
95 | return zipStr.join("");
96 | }
97 |
98 | function isDigit(c) {
99 |     return c >= "0" && c <= "9";
100 | }
101 |
102 | function isLower(c) {
```

```
103 | return c >= "a" && c <= "z";
    | 104 | }
```

Java算法源码

```
1 import java.util.Scanner;
2
3 public class Main {
4     public static void main(String[] args) {
5         Scanner sc = new Scanner(System.in);
6
7         String zipStr = sc.nextLine(); // 输入的压缩串
8         String unZipStr = unZip(zipStr); // 解压后字符串
9
10        if ("!error".equals(unZipStr) && !zip(unZipStr).equals(zipStr)) { // 对解压后的字符串二次压缩，如果二次压缩结果和输入的压缩串不一致，则输入不合法
11            System.out.println("!error");
12        } else {
13            System.out.println(unZipStr);
14        }
15    }
16
17    // 解压
18    public static String unZip(String zipStr) {
19        char[] s = zipStr.toCharArray(); // charAt写出来的代码太臃肿了
20
21        StringBuilder unZipStr = new StringBuilder(); // 记录解压结果串
22
23        int i = 0;
24        while (i < s.length) {
25            char c = s[i];
26
27            if (Character.isDigit(c)) { // 如果遍历的字符是数字
28                // 则探索出之后连续的所有数字
29                StringBuilder num = new StringBuilder();
30                while (i < s.length && Character.isDigit(s[i])) {
```

```
31         num.append(s[i]); 32         i++;
32     }
33
34
35     // 以及跟着的一个小写字母
36     if (i < s.length && Character.isLowerCase(s[i])) {
37         int repeatCount = Integer.parseInt(num.toString());
38         char repeatLetter = s[i];
39
40         // 必须超过连续两个相同小写字母才能进行压缩
41         if (repeatCount <= 2) return "!error";
42
43         // 解压
44         for (int k = 0; k < repeatCount; k++) {
45             unZipStr.append(repeatLetter);
46         }
47     } else {
48         // 数字后面没有跟着小写字母，则不合法
49         return "!error";
50     }
51
52     } else if (Character.isLowerCase(c)) {
53         // 单独的小写字母直接并入解压串
54         unZipStr.append(c);
55     } else {
56         // 压缩串中含有数字,小写字母以外的字符，则不合法
57         return "!error";
58     }
59
60     i++;
61 }
62
63 return unZipStr.toString();
64 }
65
66 // 压缩
67 public static String zip(String unZipStr) {
68     StringBuilder zipStr = new StringBuilder();
69
```

```
70 // 加一个空格不影响压缩结果, 同时可以避免收尾操作
71
72
73 // 记录上一个字母, 以及它的出现次数
74 char last = unZipStr.charAt(0);
75 int count = 1;
76
77 for (int i = 1; i < unZipStr.length(); i++) {
78     char curt = unZipStr.charAt(i);
79
80     if (curt == last) {
81         // 当前字母和上一个字母相同, 则连续相同该字母数量++
82         count++;
83     } else {
84         // 当前字母和上一个字母不相同, 则连续相同字母被打断
85         if (count > 2) {
86             // 超过两个连续相同字母, 则进行压缩
87             zipStr.append(count);
88             zipStr.append(last);
89         } else if (count == 2) {
90             // 两个连续相同字母
91             zipStr.append(last);
92             zipStr.append(last);
93         } else {
94             // 一个字母
95             zipStr.append(last);
96         }
97
98         // 更新last和count
99         last = curt;
100        count = 1;
101    }
102 }
103
104 return zipStr.toString();
105 }
106 }
```


Python算法源码

```
1 def unZip(zipStr):
2     unZipStr = [] # 记录解压结果串
3
4     i = 0
5     while i < len(zipStr):
6         c = zipStr[i]
7
8         if c.isdigit(): # 如果遍历的字符是数字
9             # 则探索出之后连续的所有数字
10            num = []
11            while i < len(zipStr) and zipStr[i].isdigit():
12                num.append(zipStr[i])
13                i += 1
14
15            # 以及跟着的一个小写字母
16            if i < len(zipStr) and zipStr[i].islower():
17                repeatCount = int("".join(num))
18                repeatLetter = zipStr[i]
19
20                # 必须超过连续两个相同小写字母才能进行压缩
21                if repeatCount <= 2:
22                    return "!error"
23
24                # 解压
25                unZipStr.append("".join([repeatLetter] * repeatCount))
26            else:
27                # 数字后面没有跟着小写字母, 则不合法
28                return "!error"
29        elif c.islower():
30            # 单独的小写字母直接并入解压串
31            unZipStr.append(c)
32        else:
33            # 压缩串中含有数字, 小写字母以外的字符, 则不合法
34            return "!error"
35
36        i += 1
```

```
37 |
38 |     return "".join(unZipStr)
39 |
40 |
41 | def zip(unZipStr):
42 |     zipStr = []
43 |
44 |     # 加一个空格不影响压缩结果，同时可以避免收尾操作
45 |     unZipStr += " "
46 |
47 |     # 记录上一个字母，以及它的出现次数
48 |     last = unZipStr[0]
49 |     count = 1
50 |
51 |     for i in range(1, len(unZipStr)):
52 |         cur = unZipStr[i]
53 |
54 |         if cur == last:
55 |             # 当前字母和上一个字母相同，则连续相同该字母数量++
56 |             count += 1
57 |         else:
58 |             # 当前字母和上一个字母不相同，则连续相同字母被打断
59 |             if count > 2:
60 |                 # 超过两个连续相同字母，则进行压缩
61 |                 zipStr.append(str(count))
62 |                 zipStr.append(last)
63 |             else:
64 |                 # 不超过两个连续相同字母，则不进行压缩
65 |                 zipStr.append("".join([last] * count))
66 |
67 |             # 更新last和count
68 |             last = cur
69 |             count = 1
70 |
71 |     return "".join(zipStr)
72 |
73 |
74 | if __name__ == '__main__':
75 |     zipStr = input() # 输入的压缩串
```

```

76 | unzipStr = unzip(zipStr) # 解压后字符串
77 |
78 | if "!error" == unzipStr or zipStr != zip(unzipStr): # 对解压后的字符串二次压缩, 如果二次压缩结果和输入的压缩串不一致, 则输入不合法
79 |     print("!error")
80 | else:
81 |     print(unzipStr)

```

C算法源码

```

1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <stdbool.h>
5 |
6 | #define MAX_LEN 10000
7 |
8 | bool isDigit(char c) {
9 |     return c >= '0' && c <= '9';
10 | }
11 |
12 | bool isLower(char c) {
13 |     return c >= 'a' && c <= 'z';
14 | }
15 |
16 | char *unzip(char *zipStr) {
17 |     // 记录解压结果串
18 |     char *unzipStr = (char *) calloc(MAX_LEN, sizeof(char));
19 |
20 |     int i = 0;
21 |     while (i < strlen(zipStr)) {
22 |         char c = zipStr[i];
23 |
24 |         if (isDigit(c)) { // 如果遍历的字符是数字
25 |             // 则探索出之后连续的所有数字
26 |             char num[MAX_LEN] = {'\0'};
27 |             int num_size = 0;

```

```

28 |         while (i < strlen(zipStr) && isDigit(zipStr[i])) { 29 |             num[num_size++] = zipStr[i];
29 |             i++;
30 |         }
31 |
32 |
33 |         // 以及跟着的一个小写字母
34 |         if (i < strlen(zipStr) && isLower(zipStr[i])) {
35 |             int repeatCount = atoi(num);
36 |             char repeatLetter = zipStr[i];
37 |
38 |             // 必须超过连续两个相同小写字母才能进行压缩
39 |             if (repeatCount <= 2) {
40 |                 return "!error";
41 |             }
42 |
43 |             // 解压
44 |             for (int k = 0; k < repeatCount; k++) {
45 |                 unZipStr[strlen(unZipStr)] = repeatLetter;
46 |             }
47 |         } else {
48 |             // 数字后面没有跟着小写字母，则不合法
49 |             return "!error";
50 |         }
51 |
52 |     } else if (isLower(c)) {
53 |         // 单独的小写字母直接并入解压串
54 |         unZipStr[strlen(unZipStr)] = c;
55 |     } else {
56 |         // 压缩串中含有数字,小写字母以外的字符，则不合法
57 |         return "!error";
58 |     }
59 |
60 |     i++;
61 | }
62 |
63 | return unZipStr;
64 | }
65 |
66 | char *zip(char *unZipStr) {

```

```
67 char *zipStr = (char *) calloc(MAX_LEN, sizeof(char)); 68
69 // 加一个空格不影响压缩结果, 同时可以避免收尾操作
70 unzipStr[strlen(unzipStr)] = ' ';
71
72 // 记录上一个字母, 以及它的出现次数
73 char last = unzipStr[0];
74 int count = 1;
75
76 for (int i = 1; i < strlen(unzipStr); i++) {
77     char cur = unzipStr[i];
78
79     if (cur == last) {
80         // 当前字母和上一个字母相同, 则连续相同该字母数量++
81         count++;
82     } else {
83         // 当前字母和上一个字母不相同, 则连续相同字母被打断
84         if (count > 2) {
85             // 超过两个连续相同字母, 则进行压缩
86             char tmp[MAX_LEN];
87             sprintf(tmp, "%d", count);
88             strcat(zipStr, tmp);
89
90             zipStr[strlen(zipStr)] = last;
91         } else if (count == 2) {
92             // 两个连续相同字母
93             zipStr[strlen(zipStr)] = last;
94             zipStr[strlen(zipStr)] = last;
95         } else {
96             // 一个字母
97             zipStr[strlen(zipStr)] = last;
98         }
99
100         // 更新last和count
101         last = cur;
102         count = 1;
103     }
104 }
105
```

```

106     return zipStr;
107 }
108
109 int main() {
110     char zipStr[MAX_LEN]; // 输入的压缩串
111     scanf("%s", zipStr);
112
113     char *unZipStr = unzip(zipStr); // 解压后字符串
114
115     if (strcmp("!error", unZipStr) == 0 ||
116         strcmp(zipStr, zip(unZipStr)) != 0) { // 对解压后的字符串二次压缩，如果二次压缩结果和输入的压缩串不一致，则输入不合法
117         puts("!error");
118     } else {
119         puts(unZipStr);
120     }
121
122     return 0;
123 }

```

C++ 算法源码

```

1  #include <bits/stdc++.h>
2
3  using namespace std;
4
5  // 解压
6  string unzip(string &s) {
7      string unZipStr; // 记录解压结果串
8
9      int i = 0;
10     while (i < s.length()) {
11         char c = s[i];
12
13         if (isdigit(c)) { // 如果遍历的字符是数字
14             // 则探索出之后连续的所有数字
15             string num;

```

```
16         while (i < s.length() && isdigit(s[i])) { 17 |           num += s[i];
17             i++;
18         }
19
20
21         // 以及跟着的一个小写字母
22         if (i < s.length() && islower(s[i])) {
23             int repeatCount = stoi(num);
24             char repeatLetter = s[i];
25
26             // 必须超过连续两个相同小写字母才能进行压缩
27             if (repeatCount <= 2) return "!error";
28
29             // 解压
30             for (int k = 0; k < repeatCount; k++) {
31                 unZipStr += repeatLetter;
32             }
33         } else {
34             // 数字后面没有跟着小写字母, 则不合法
35             return "!error";
36         }
37
38     } else if (islower(c)) {
39         // 单独的小写字母直接并入解压串
40         unZipStr += c;
41     } else {
42         // 压缩串中含有数字, 小写字母以外的字符, 则不合法
43         return "!error";
44     }
45
46     i++;
47 }
48
49 return unZipStr;
50 }
51
52 // 压缩
53 string zip(string &s) {
54     string zipStr;
```

```
55 |
56 | // 加一个空格不影响压缩结果，同时可以避免收尾操作
57 | s += " ";
58 |
59 | // 记录上一个字母，以及它的出现次数
60 | char last = s[0];
61 | int count = 1;
62 |
63 | for (int i = 1; i < s.length(); i++) {
64 |     char cur = s[i];
65 |
66 |     if (cur == last) {
67 |         // 当前字母和上一个字母相同，则连续相同该字母数量++
68 |         count++;
69 |     } else {
70 |         // 当前字母和上一个字母不相同，则连续相同字母被打断
71 |         if (count > 2) {
72 |             // 超过两个连续相同字母，则进行压缩
73 |             zipStr += to_string(count);
74 |             zipStr += last;
75 |         } else if (count == 2) {
76 |             // 两个连续相同字母
77 |             zipStr += last;
78 |             zipStr += last;
79 |         } else {
80 |             // 一个字母
81 |             zipStr += last;
82 |         }
83 |
84 |         // 更新last和count
85 |         last = cur;
86 |         count = 1;
87 |     }
88 | }
89 |
90 | return zipStr;
91 | }
92 |
93 | int main() {
```



```
94 | string zipStr; // 输入的压缩串
    |                                     95 | cin >> zipStr;
96 |
97 | string unZipStr = unZip(zipStr); // 解压后字符串
98 |
99 | if ("!error" == unZipStr || zipStr != zip(unZipStr)) { // 对解压后的字符串二次压缩, 如果二次压缩结果和输入的压缩串不一致, 则输入不合法
100 |     cout << "!error" << endl;
101 | } else {
102 |     cout << unZipStr << endl;
103 | }
104 |
105 | return 0;
106 | }
```